

Informe: Implementación de Sistema de Agencias

Lorenzo Stefano

Abril 2025 - IIC2133

1. Explicación General

Para abordar el problema de gestión de agencias y agentes, implementé un sistema basado en un **Max Heap** como estructura de datos principal. La elección de esta estructura se justifica por la necesidad de acceder eficientemente al agente de mayor prioridad (más alto nivel) en cada operación MISION.

1.1. Estructuras Principales

- **Agent (heap.h, línea 5)**: Estructura que representa un agente con identificador único y nivel.
- **MaxHeap (heap.h, línea 10)**: Implementa un heap máximo utilizando un arreglo dinámico.
- **Agency (agency.h, línea 7)**: Contiene un heap de agentes disponibles y metadatos.

1.2. Enfoque General

La implementación utiliza un enfoque modular donde:

- Las operaciones básicas del heap (inserción, extracción del máximo) están aisladas en `heap.c`.
- Las operaciones de alto nivel relacionadas con agencias (BUILD, MISSION, etc.) están implementadas en `agency.c`.
- El programa principal en `main.c` procesa los comandos y llama a las funciones correspondientes.

Un aspecto clave de la implementación es la función de comparación de agentes (`compare_agents` en `heap.c`, línea 15) que determina la prioridad en el heap basándose primero en el nivel y luego en el ID cuando hay niveles iguales:

```
1 int compare_agents(const Agent* a1, const Agent* a2) {
2     if (a1->level != a2->level) {
3         return a1->level - a2->level; // Mayor nivel tiene mayor prioridad
4     }
5     // Mismo nivel, menor ID tiene mayor prioridad
6     return a2->id - a1->id;
7 }
```

Esta función es fundamental para mantener la propiedad de heap y garantizar que siempre se extraiga el agente correcto según las especificaciones del problema.

2. Complejidad

A continuación, se justifica la complejidad de cada operación clave del sistema:

2.1. BUILD - $O(N \log N)$

La operación BUILD (implementada en `handle_build`, `agency.c`, línea 87) construye una agencia con N agentes insertándolos uno por uno en el heap:

- Se crea un heap vacío: $O(1)$
- Para cada uno de los N agentes, se realiza una inserción en el heap: $O(\log N)$ por inserción
- Complejidad total: $O(N \log N)$

Se podría haber implementado una construcción de heap más eficiente con complejidad $O(N)$ usando `build_max_heap_from_array` (`heap.c`, línea 152), pero la consigna especificaba utilizar inserciones repetidas.

2.2. MISION - $O(\log N)$

La operación MISION (implementada en `handle_mission`, `agency.c`, línea 219) extrae el agente de mayor prioridad:

- Extracción del máximo del heap (`heap_extract_max`): $O(\log N)$
- Esta operación implica extraer la raíz y reorganizar el heap

2.3. STATUS - $O(S)$

La operación STATUS (implementada en `handle_status`, `agency.c`, línea 245) recorre todas las agencias existentes:

- Recorre el arreglo de agencias una vez: $O(S)$ donde S es el número máximo posible de agencias
- Para cada agencia, obtiene el líder con `heap_peek_max`: $O(1)$
- Complejidad total: $O(S)$

2.4. RETURN - $O(\log N)$

La operación RETURN (implementada en `handle_return`, `agency.c`, línea 273) inserta un agente que regresa a una agencia:

- Inserción en el heap (`heap_insert`): $O(\log N)$
- Esta operación implica insertar al final y reorganizar el heap hacia arriba

2.5. UPDATE_LEVEL - $O(N)$

La operación UPDATE_LEVEL (implementada en `handle_update_level`, `agency.c`, línea 315) actualiza el nivel de un agente existente:

- Búsqueda lineal del agente por ID: $O(N)$
- Actualización del nivel y reorganización del heap: $O(\log N)$
- Complejidad total: $O(N)$ dominada por la búsqueda

Si bien se podría optimizar utilizando una estructura de índice adicional para localizar agentes en $O(1)$, la especificación requería que UPDATE_LEVEL fuera $O(N)$.

2.6. MERGE - $O((N+M) \log (N+M))$

La operación MERGE (implementada en `handle_merge`, `agency.c`, línea 365) combina dos agencias:

- Creación de un nuevo heap: $O(1)$
- Inserción de N agentes de la primera agencia: $O(N \log(N+M))$
- Inserción de M agentes de la segunda agencia: $O(M \log(N+M))$
- Complejidad total: $O((N+M) \log(N+M))$

La complejidad teórica podría optimizarse a $O(N+M)$ utilizando un enfoque de mezcla directa de heaps, pero la implementación actual cumple con la especificación.

3. Conclusión

La implementación presentada utiliza estructuras de datos y algoritmos adecuados para cumplir con las complejidades requeridas en el enunciado. El uso de un Max Heap como estructura central permite realizar eficientemente las operaciones de prioridad, mientras que el diseño modular facilita la extensión y mantenimiento del código.

Las funciones auxiliares `heapify_up` y `heapify_down` (`heap.c`, líneas 42 y 21 respectivamente) son fundamentales para mantener la propiedad de heap después de inserciones y extracciones, garantizando las complejidades logarítmicas esperadas.